

```
In [ ]: Alexis Root
      ### Project 01
```

```
In [ ]: 1) What is the salary range for a lead data science position, and what are the diff
      for an offshore employee? How are the changing company size and current economic co
      the salary range for this?

      2) How can we establish a clear, bias-free salary range that attracts top-tier tale
      internal equity as we scale from a small company to a mid-sized organization?
      *2 Developed using Google Gen AI input, asked for a different way to ask the questi
      the best fitting version that AI gave
```

```
In [ ]: I think the part that the AI mentioned about focusing on the current company scale
      should be added to the overall question. A small company that is growing may benefi
      employee with more overall experience to help drive the projects as they mentioned
      a team, once the business grows further and they have the budget to hire more peopl
      costly to higher a lower cost employee with less experience and then have to hire s
      suppliment what they can do.
      I think the part about the offshore employee is less relevant in regards to the sala
      is a significant difference found, or unless the employee has significantly higher
      an employee closer to the company iteself is better in some ways since they would b
      and could more easily travel as needed to the office. Other than that, the location
      more or less irrelevant to me.
```

```
In [1]: import pandas as pd
      import numpy as np
      import seaborn as sns
      import plotnine as p9
      import matplotlib.pyplot as plt
      import matplotlib.ticker as mtick
      from scipy.stats import ttest_ind
```

```
In [2]: infile="project_1_data.csv"

      comp_data_df=pd.read_csv(infile, index_col=0)
```

```
In [3]: comp_data_df.dtypes
```

```
Out[3]: work_year          int64
      experience_level    object
      employment_type     object
      job_title           object
      salary              int64
      salary_currency     object
      salary_in_usd       int64
      employee_residence  object
      remote_ratio        int64
      company_location    object
      company_size        object
      dtype: object
```

```
In [17]: #Updating Data Types
      my_dtypes = {
```

```

        "experience_level": "category",
        "work_year": int,
        "employment_type": "category",
        "job_title": "category",
        "salary_currency": "category",
        "employee_residence": "category",
        "company_location": "category",
        "company_size": "category",
    }

infile = "project_1_data.csv"
comp_data_df = pd.read_csv(infile, dtype=my_dtypes, index_col=0)
# Ordering experience level
comp_data_df["experience_level"] = pd.Categorical(
    comp_data_df["experience_level"].map({
        "EN": "Entry Level",
        "MI": "Mid Level",
        "SE": "Senior Level",
        "EX": "Executive Level"
    }),
    categories=[
        "Entry Level",
        "Mid Level",
        "Senior Level",
        "Executive Level"
    ],
    ordered=True
)
# Ordering Remote
comp_data_df["remote_ratio"] = pd.Categorical(
    comp_data_df["remote_ratio"].map({
        0: "Onsite",
        50: "Hybrid",
        100: "Fully Remote",
    }),
    categories=[
        "Onsite",
        "Hybrid",
        "Fully Remote"
    ],
    ordered=True
)
# Ordering company size (S, M, L)
comp_data_df["company_size"] = pd.Categorical(
    comp_data_df["company_size"].map({
        "S": "Small",
        "M": "Medium",
        "L": "Large"
    }),
    categories=[
        "Small",
        "Medium",
        "Large"
    ],
    ordered=True
)

```

```
In [18]: comp_data_df.dtypes
```

```
Out[18]: work_year          int64
experience_level  category
employment_type  category
job_title        category
salary           int64
salary_currency  category
salary_in_usd    int64
employee_residence category
remote_ratio     category
company_location category
company_size     category
dtype: object
```

```
In [19]: type(comp_data_df)
```

```
Out[19]: pandas.core.frame.DataFrame
```

```
In [20]: comp_data_df.head
```

```
Out[20]: <bound method NDFrame.head of
job_title \
0      2020      Mid Level      FT      Data Scientist
1      2020      Senior Level      FT      Machine Learning Scientist
2      2020      Senior Level      FT      Big Data Engineer
3      2020      Mid Level      FT      Product Data Analyst
4      2020      Senior Level      FT      Machine Learning Engineer
..      ...      ...      ...      ...
602     2022      Senior Level      FT      Data Engineer
603     2022      Senior Level      FT      Data Engineer
604     2022      Senior Level      FT      Data Analyst
605     2022      Senior Level      FT      Data Analyst
606     2022      Mid Level      FT      AI Scientist

      salary salary_currency salary_in_usd employee_residence remote_ratio \
0      70000      EUR      79833      DE      Onsite
1     260000      USD      260000      JP      Onsite
2      85000      GBP      109024      GB      Hybrid
3      20000      USD      20000      HN      Onsite
4     150000      USD      150000      US      Hybrid
..      ...      ...      ...      ...
602    154000      USD      154000      US      Fully Remote
603    126000      USD      126000      US      Fully Remote
604    129000      USD      129000      US      Onsite
605    150000      USD      150000      US      Fully Remote
606    200000      USD      200000      IN      Fully Remote

      company_location company_size
0      DE      Large
1      JP      Small
2      GB      Medium
3      HN      Small
4      US      Large
..      ...      ...
602     US      Medium
603     US      Medium
604     US      Medium
605     US      Medium
606     US      Large
```

[607 rows x 11 columns]>

```
In [21]: comp_data_df.describe()
#work_year- data range is from 2020-2022. Good to know, summary data hear other tha
#salary- Numbers are all over the place because the salaries in this column are all
#salary_in_usd- min=0 is concerning there should be no $0 salary. Row needs to be r
#remote_ratio- max= full time remote or onsite, min= onsite or fully remote, mean b
#No other variables listed on table because they are defined as category types not
```

Out[21]:

	work_year	salary	salary_in_usd
count	607.000000	6.070000e+02	607.000000
mean	2021.405272	3.240001e+05	112297.869852
std	0.692133	1.544357e+06	70957.259411
min	2020.000000	4.000000e+03	2859.000000
25%	2021.000000	7.000000e+04	62726.000000
50%	2022.000000	1.150000e+05	101570.000000
75%	2022.000000	1.650000e+05	150000.000000
max	2022.000000	3.040000e+07	600000.000000

In [22]:

```
comp_data_df.info()
#no null values confirmed, data types are set as I need
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 607 entries, 0 to 606
Data columns (total 11 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   work_year              607 non-null    int64
 1   experience_level        607 non-null    category
 2   employment_type         607 non-null    category
 3   job_title               607 non-null    category
 4   salary                  607 non-null    int64
 5   salary_currency         607 non-null    category
 6   salary_in_usd           607 non-null    int64
 7   employee_residence      607 non-null    category
 8   remote_ratio            607 non-null    category
 9   company_location        607 non-null    category
10   company_size            607 non-null    category
dtypes: category(8), int64(3)
memory usage: 32.4 KB
```

In [23]:

```
comp_data_df.loc[comp_data_df["employment_type"] == "FT",
                  ["salary_in_usd", "remote_ratio", "experience_level"]].groupby(["e
#Salaries are going up as you move upwards in experience level as expected. People
```

Out[23]:

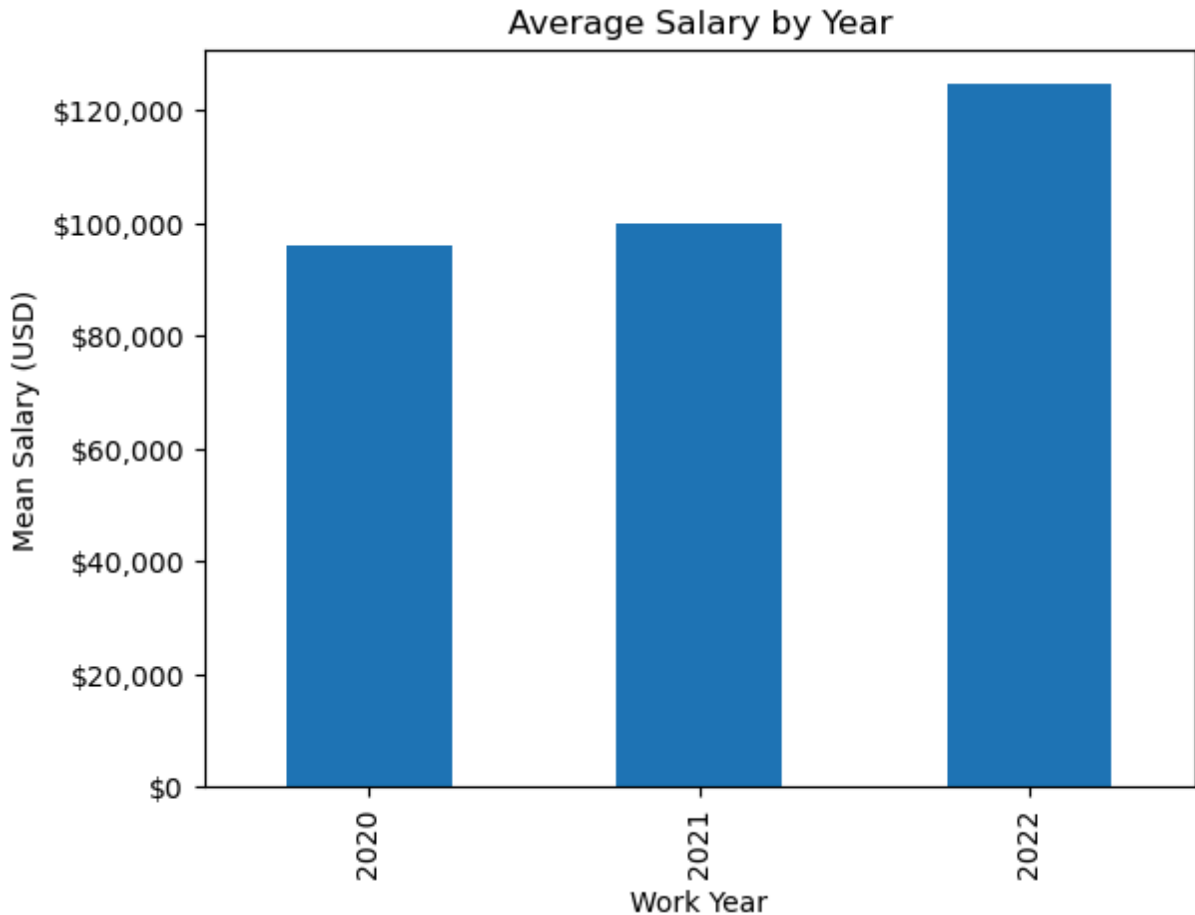
		salary_in_usd
experience_level	remote_ratio	
Entry Level	Onsite	57666.357143
	Hybrid	63897.619048
	Fully Remote	66885.454545
Mid Level	Onsite	83259.327273
	Hybrid	80486.692308
	Fully Remote	93685.794643
Senior Level	Onsite	141177.759259
	Hybrid	95250.888889
	Fully Remote	144428.776650
Executive Level	Onsite	158948.666667
	Hybrid	143175.400000
	Fully Remote	210321.764706

In [24]: `comp_data_df.loc[comp_data_df["employment_type"] == "FT",
["salary_in_usd", "company_size", "experience_level"]].groupby(["e
#Same trend in salaries increasing as experience level increases. You see larger ga`

Out[24]:

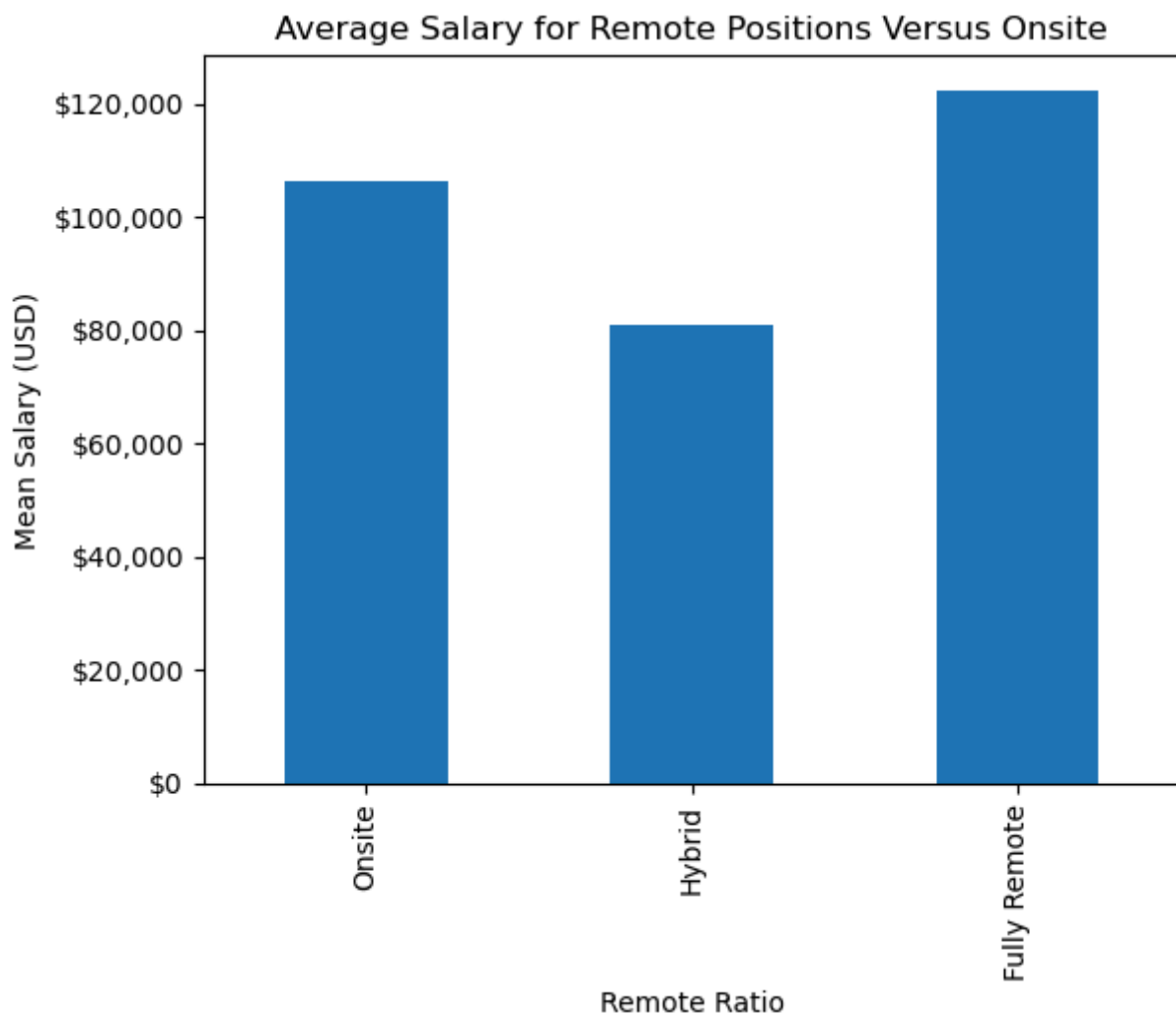
		salary_in_usd
experience_level	company_size	
Entry Level	Small	69005.200000
	Medium	52131.730769
	Large	71842.285714
Mid Level	Small	51159.379310
	Medium	91700.178947
	Large	97755.048780
Senior Level	Small	118694.666667
	Medium	137992.978378
	Large	147591.013889
Executive Level	Small	93964.000000
	Medium	178241.750000
	Large	221942.181818

```
In [25]: #Graph for Work Year variable
ax = comp_data_df.groupby("work_year")["salary_in_usd"].mean().plot(kind="bar")
ax.yaxis.set_major_formatter(mtick.StrMethodFormatter('${x:,.0f}')) # Formats y-axis
plt.title("Average Salary by Year")
plt.xlabel("Work Year")
plt.ylabel("Mean Salary (USD)")
plt.show()
#salaries are going up year after year as expected, salaries did go up more between
```



```
In [26]: #Graph for Remote- Ratio variable
ax = comp_data_df.groupby("remote_ratio")["salary_in_usd"].mean().plot(kind="bar")
ax.yaxis.set_major_formatter(mtick.StrMethodFormatter('${x:,.0f}')) # Formats y-axis
plt.title("Average Salary for Remote Positions Versus Onsite")
plt.xlabel("Remote Ratio")
plt.ylabel("Mean Salary (USD)")
plt.show()
#This is not what I would expect, there could be other factors impacting this. Such
# It looks like a 100% remote position pays the highest over a 0% remote position.
#I would expect 50% to be the middle salary range but it is the lowest salary range.
```

C:\Users\Lex\AppData\Local\Temp\ipykernel_15692\1345826130.py:2: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

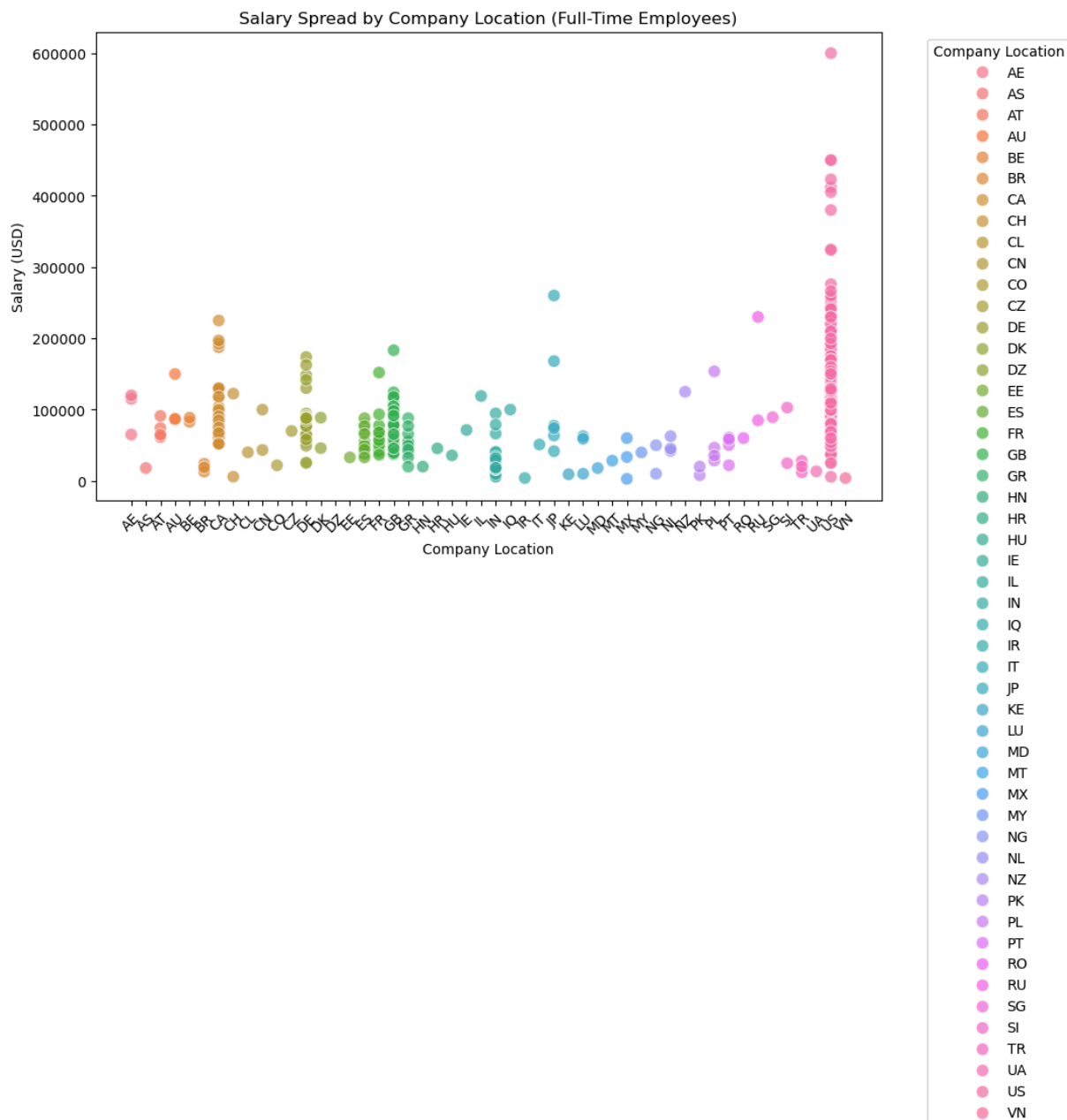


```
In [27]: #Graphing the mean salaries for different employee types- FT/PT/FL/contractor
mean_salary = comp_data_df.groupby("employment_type", observed=True)["salary_in_usd"]
fig, ax = plt.subplots(figsize=(6,4))
mean_salary.plot(kind="bar", ax=ax)
ax.yaxis.set_major_formatter(mtick.StrMethodFormatter('${x:,.0f}')) # Format y-axis
ax.set_title("Salary Ranges by Position Type")
ax.set_xlabel("Position Type")
ax.set_ylabel("Mean Salary (USD)")
plt.show()

#Contractors have the highest average salary compared to a full time position. Base
#FL workers are only hired on for one project at a time, so their salary being low
#PT worker salaries being low makes sense although under $50K seems a bit low depen
#Plan to not include FL/PT/Contractors in calculations since PT/FL is not what we a
```

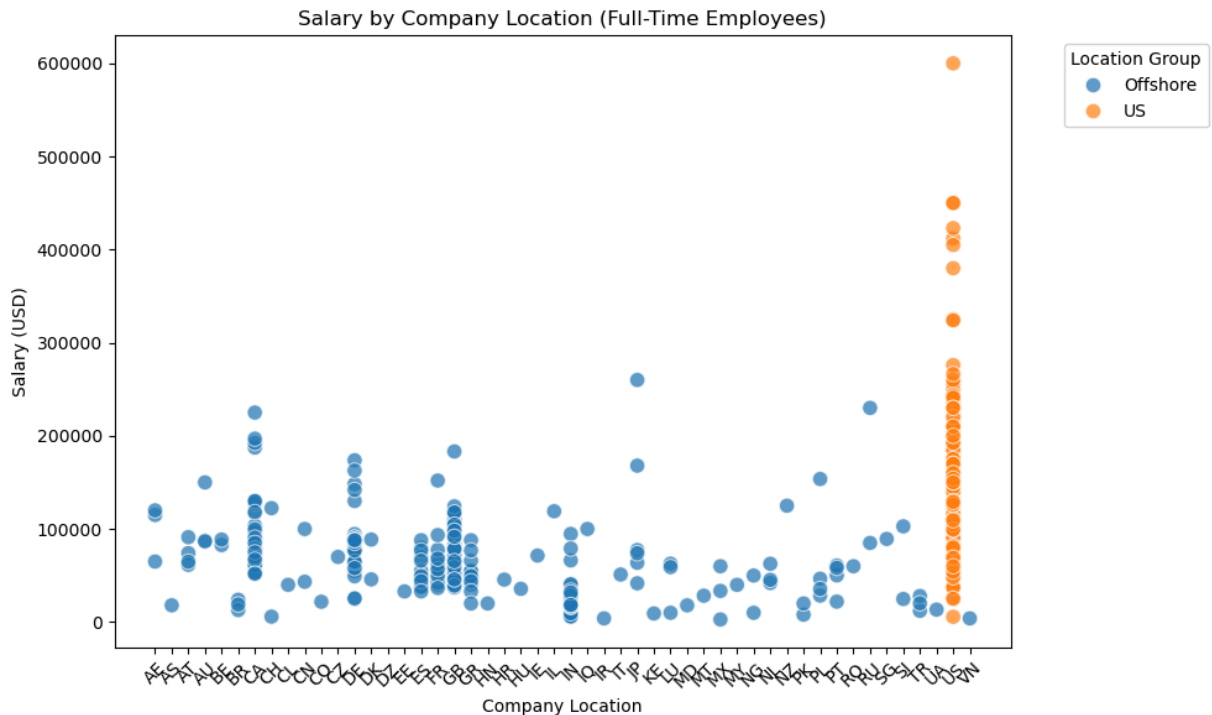


```
In [18]: #Scatterplot for USD salary differences across different company Locations
df = comp_data_df[comp_data_df['employment_type'] == 'FT']
plt.figure(figsize=(10,6))
sns.scatterplot(
    data=df,
    x='company_location',
    y='salary_in_usd',
    hue='company_location', # different color per Location
    alpha=0.7, #controls transparency of point, 1=opaque, 0=invisible
    s=80 # size of points
)
plt.title('Salary Spread by Company Location (Full-Time Employees)')
plt.xlabel('Company Location')
plt.ylabel('Salary (USD)')
plt.xticks(rotation=45)
plt.legend(title='Company Location', bbox_to_anchor=(1.05, 1), loc='upper left') #M
plt.show()
#Hard to decipher even with the points having different colors; US is a pinkish col
#Looks like most countries have lower salaries in USD but hard to discern from culm
```

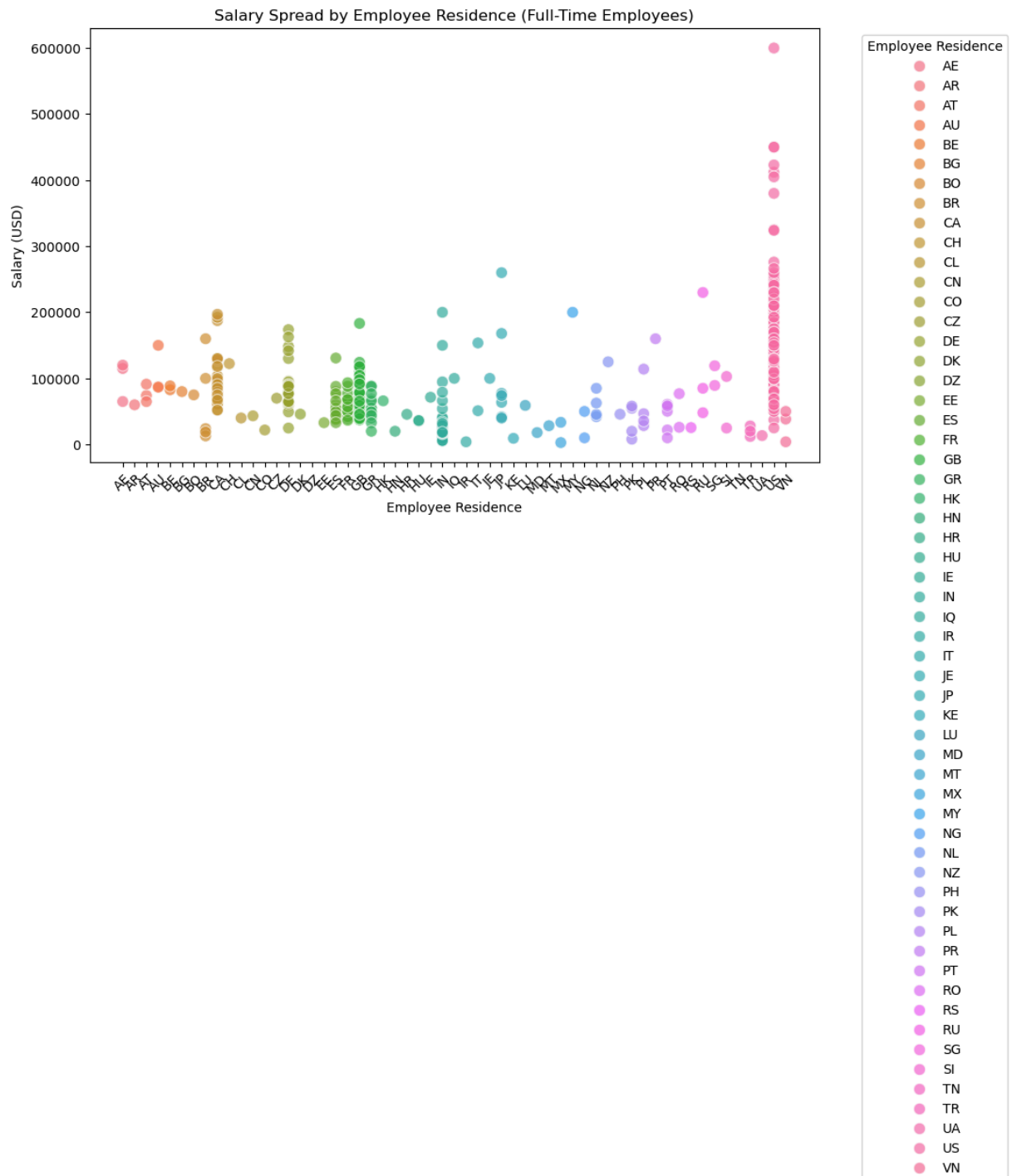


```
In [19]: #Seperating US from all other countries to see how scaterplot looks compared to the
# Create a US vs Offshore grouping
comp_data_df['location_group'] = comp_data_df['company_location'].apply(lambda x: '
#Loops through each salary_currency listing, adds either US/offshore to new col cur
df = comp_data_df[comp_data_df['employment_type'] == 'FT']
plt.figure(figsize=(10,6))
sns.scatterplot(
    data=df,
    x='company_location',
    y='salary_in_usd',
    hue='location_group', # US vs Offshore points have differennt colors
    alpha=0.7, #controls transparency of point, 1=opaque, 0=invisible
    s=80 #size of plotted points
)
plt.title('Salary by Company Location (Full-Time Employees)')
plt.xlabel('Company Location')
plt.ylabel('Salary (USD)')
```

```
plt.xticks(rotation=45) #rotate x names for visibility
plt.legend(title='Location Group', bbox_to_anchor=(1.05, 1), loc='upper left') #move legend to upper left
plt.tight_layout()
plt.show()
#Easier to see the US points, it's interesting how a majority of the points still fall
#This suggests to me that there could be a cost benefit to hiring an offshore employee
```

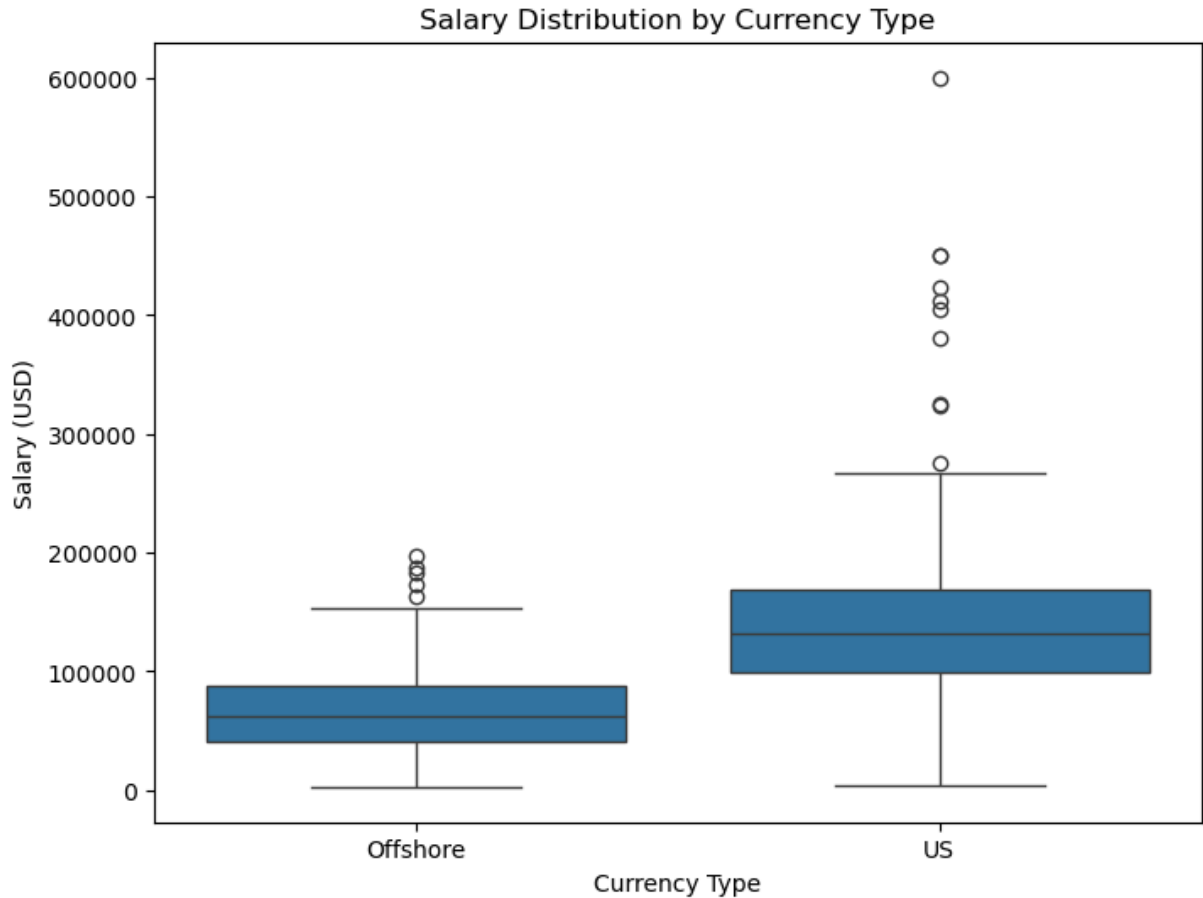


```
In [20]: #Scatterplot for USD salary differences across different employee residence locations
df = comp_data_df[comp_data_df['employment_type'] == 'FT']
plt.figure(figsize=(10,6))
sns.scatterplot(
    data=df,
    x='employee_residence',
    y='salary_in_usd',
    hue='employee_residence', # different color per location
    alpha=0.7, #controls transparency of point, 1=opaque, 0=invisible
    s=80 # size of points
)
plt.title('Salary Spread by Employee Residence (Full-Time Employees)')
plt.xlabel('Employee Residence')
plt.ylabel('Salary (USD)')
plt.xticks(rotation=45) #rotate x names for visibility
plt.legend(title='Employee Residence', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()
#Results are very similar to the company location graph.
#Despite rotating x-axis labels, still very hard to read. Not a great graph to use.
```

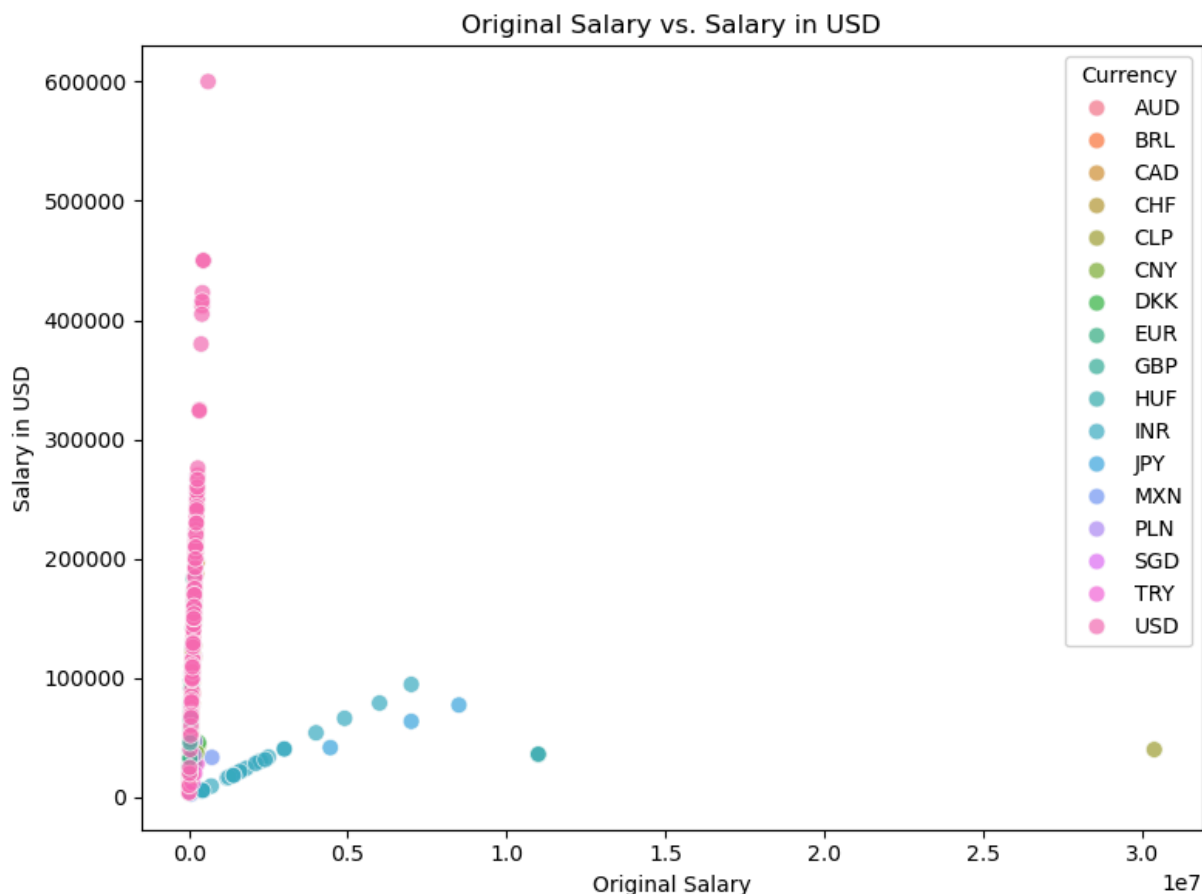


```
In [45]: #Salary Currency
comp_data_df['currency_group'] = comp_data_df['salary_currency'].apply(lambda x: 'U
df = comp_data_df[comp_data_df['employment_type'] == 'FT']
plt.figure(figsize=(8,6))
sns.boxplot(
    data=df,
    x="currency_group",
    y="salary_in_usd",
)
plt.title("Salary Distribution by Currency Type")
plt.xlabel("Currency Type")
plt.ylabel("Salary (USD)")
```

```
plt.show()
#Salaries in USD are higher for salaries paid in USD currency versus salaries paid
```



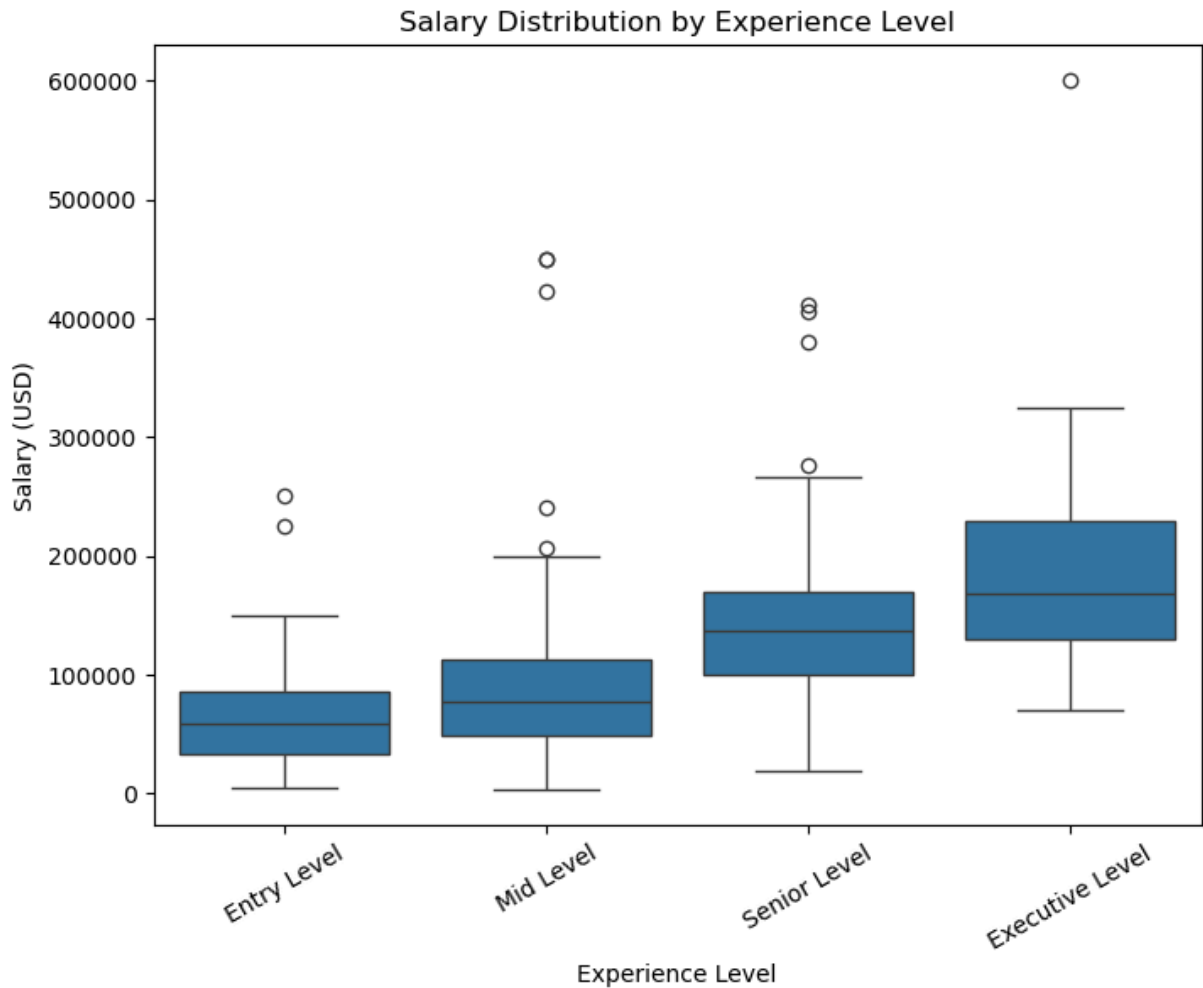
```
In [103... #Scatterplot to show relationship between salary and salary in USD
plt.figure(figsize=(8,6))
sns.scatterplot(
    data=comp_data_df,
    x='salary',
    y='salary_in_usd',
    hue='salary_currency',
    alpha=0.7,
    s=60
)
plt.title('Original Salary vs. Salary in USD')
plt.xlabel('Original Salary')
plt.ylabel('Salary in USD')
plt.legend(title='Currency')
plt.tight_layout()
plt.show()
#Hard to see much from this version of the graph, the points are all clustered toge
```



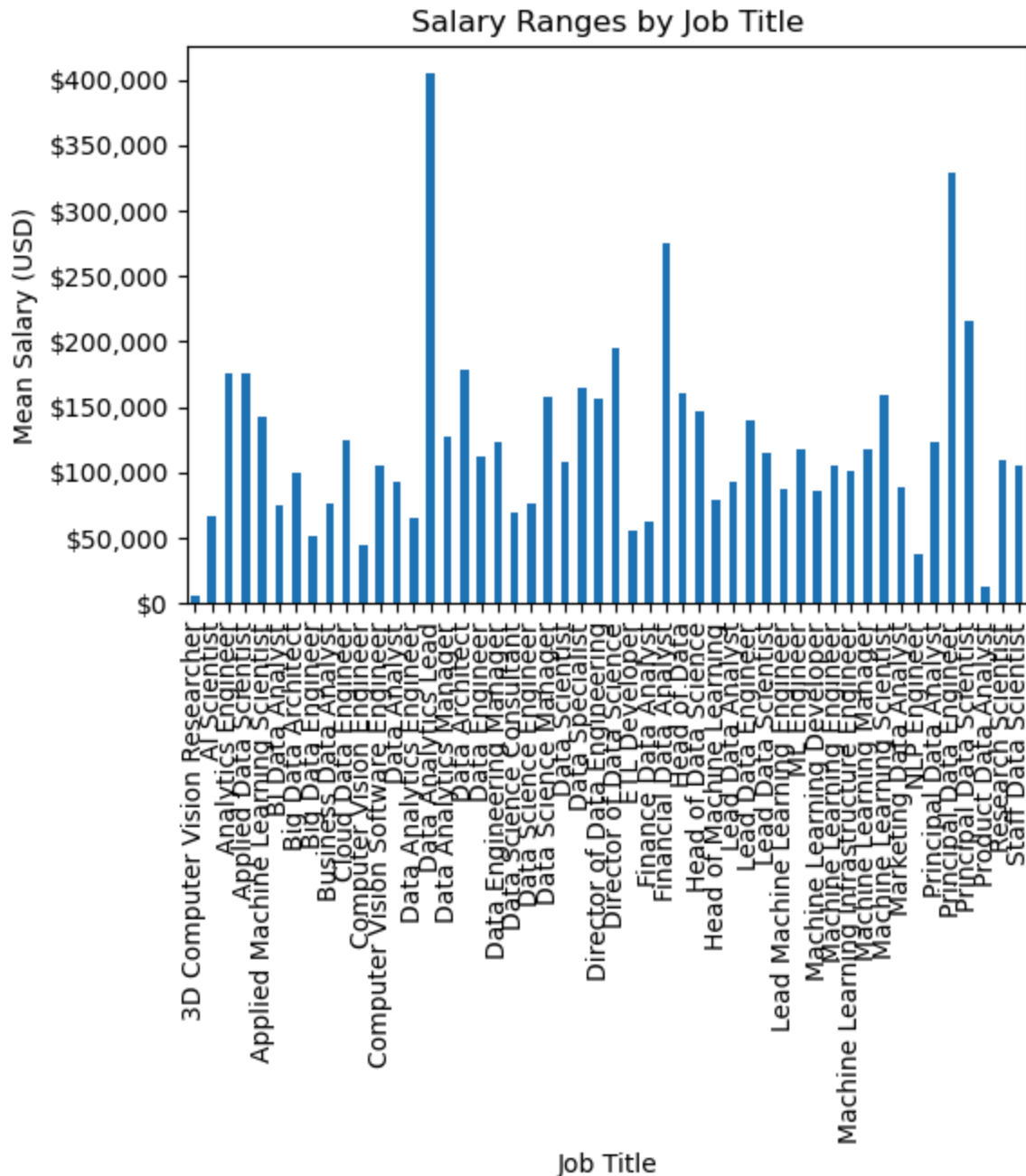
```
In [22]: #Scatterplot to show relationship between salary and salary in USD
plt.figure(figsize=(8,6))
sns.scatterplot(
    data=comp_data_df,
    x='salary',
    y='salary_in_usd',
    hue='salary_currency',
    alpha=0.7,
    s=60
)
plt.title('Original Salary vs. Salary in USD')
plt.xlabel('Original Salary')
plt.ylabel('Salary in USD')
plt.xscale('log') #transformation added to help with visibility
plt.yscale('log') #transformation added to help with visibility
plt.legend(title = 'Currency', bbox_to_anchor = (1.05, 1), loc = 'upper left') #move
plt.tight_layout()
plt.show()
#Salaries are higher in USD, the salaries in USD originally go up higher right on t
#There is a good cluster of countries aligned with a similar line as the US one sho
```



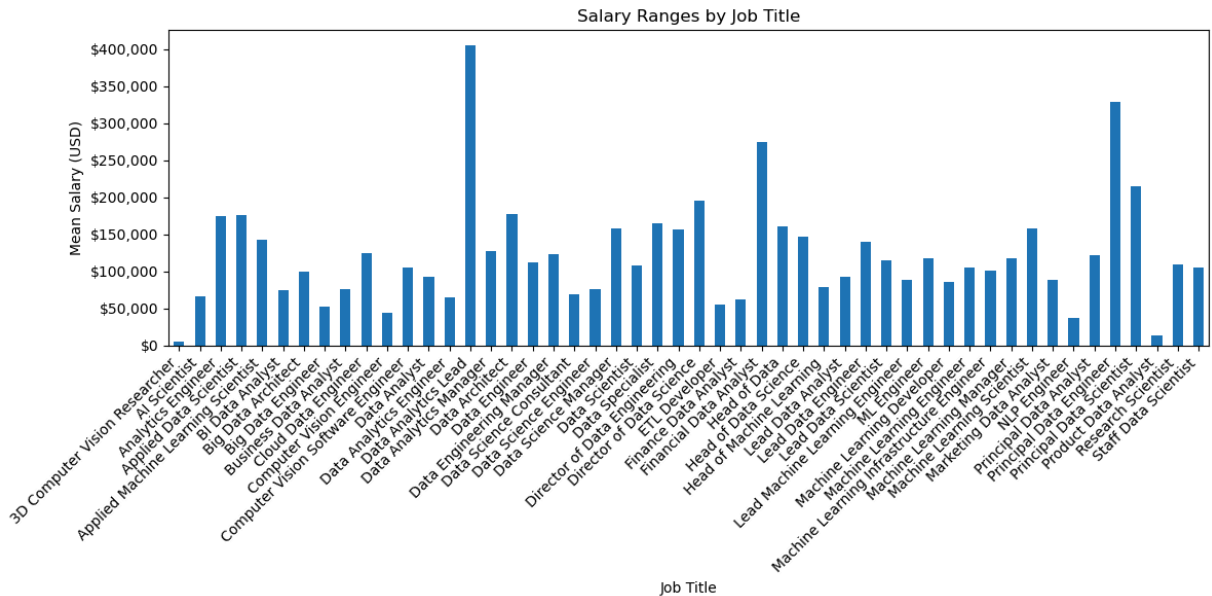
```
In [23]: #Graph of Experience Level Variable
df = comp_data_df[comp_data_df["employment_type"] == "FT"] # only full-time employ
plt.figure(figsize=(8,6))
sns.boxplot(
    data=df,
    x="experience_level",
    y="salary_in_usd",
)
plt.title("Salary Distribution by Experience Level")
plt.xlabel("Experience Level")
plt.ylabel("Salary (USD)")
plt.xticks(rotation=30) #rotated x-axis for readability, 30 degrees
plt.show()
#There's quite a few outliers here, could be between the different job titles inclu
#Salary goes up by experience as expected
#Executive level employees have the widest range of salaries within their 25-75% av
```



```
In [24]: #Initial Job Title Graph
mean_salary = comp_data_df.groupby("job_title", observed=True)["salary_in_usd"].me
fig, ax = plt.subplots(figsize=(6,4))
mean_salary.plot(kind="bar", ax=ax)
ax.yaxis.set_major_formatter(mtick.StrMethodFormatter('${x:,.0f}')) # Format y-axis
ax.set_title("Salary Ranges by Job Title")
ax.set_xlabel("Job Title")
ax.set_ylabel("Mean Salary (USD)")
plt.xticks(rotation=90) #rotates words on x-axis lables for visibility
plt.show()
#This is unreadable, not a lot can be gained from this graph
#There are too many job titles, they need to be grouped together to narrow down how
#May be able to filter out irrelevant job titles
```



```
In [47]: #Improve previous graph for wider axis
mean_salary = comp_data_df.groupby("job_title", observed=True)["salary_in_usd"].mean
# Make figure wider
fig, ax = plt.subplots(figsize=(12,6)) # Wider and taller
mean_salary.plot(kind="bar", ax=ax)
# Format y-axis as dollars
ax.yaxis.set_major_formatter(mtick.StrMethodFormatter('${x:,.0f}'))
ax.set_title("Salary Ranges by Job Title")
ax.set_xlabel("Job Title")
ax.set_ylabel("Mean Salary (USD)")
plt.xticks(rotation=45, ha='right') # Rotate 45 degrees and align right
plt.tight_layout() # Ensures everything fits without overlap
plt.show()
```



```
In [28]: #Grouping Job Titles- used AI (chat GPT) to take List of all 50 job titles in data
#Also used AI to help speed along process of writing out the individual job titles
title_grouped_comp_data = comp_data_df.copy()
# Define conditions for job groups
conditions = [
    title_grouped_comp_data['job_title'].str.contains(
        "DATA SCIENTIST|LEAD DATA SCIENTIST|PRINCIPAL DATA SCIENTIST|STAFF DATA SCI
        case=False, regex=True
    ),
    title_grouped_comp_data['job_title'].str.contains(
        "MACHINE LEARNING|ML ENGINEER|AI SCIENTIST|NLP ENGINEER|COMPUTER VISION|APP
        case=False, regex=True
    ),
    title_grouped_comp_data['job_title'].str.contains(
        "DATA ANALYST|PRODUCT DATA ANALYST|BUSINESS DATA ANALYST|MARKETING DATA ANA
        case=False, regex=True
    ),
    title_grouped_comp_data['job_title'].str.contains(
        "DATA ENGINEER|LEAD DATA ENGINEER|DATA SCIENCE ENGINEER|DATA ARCHITECT|PRIN
        case=False, regex=True
    )
]

# Job grouping Labels
choices = [
    "Data Scientist",
    "Machine Learning / AI Engineer",
    "Data Analyst",
    "Data Engineer / Architect"
]

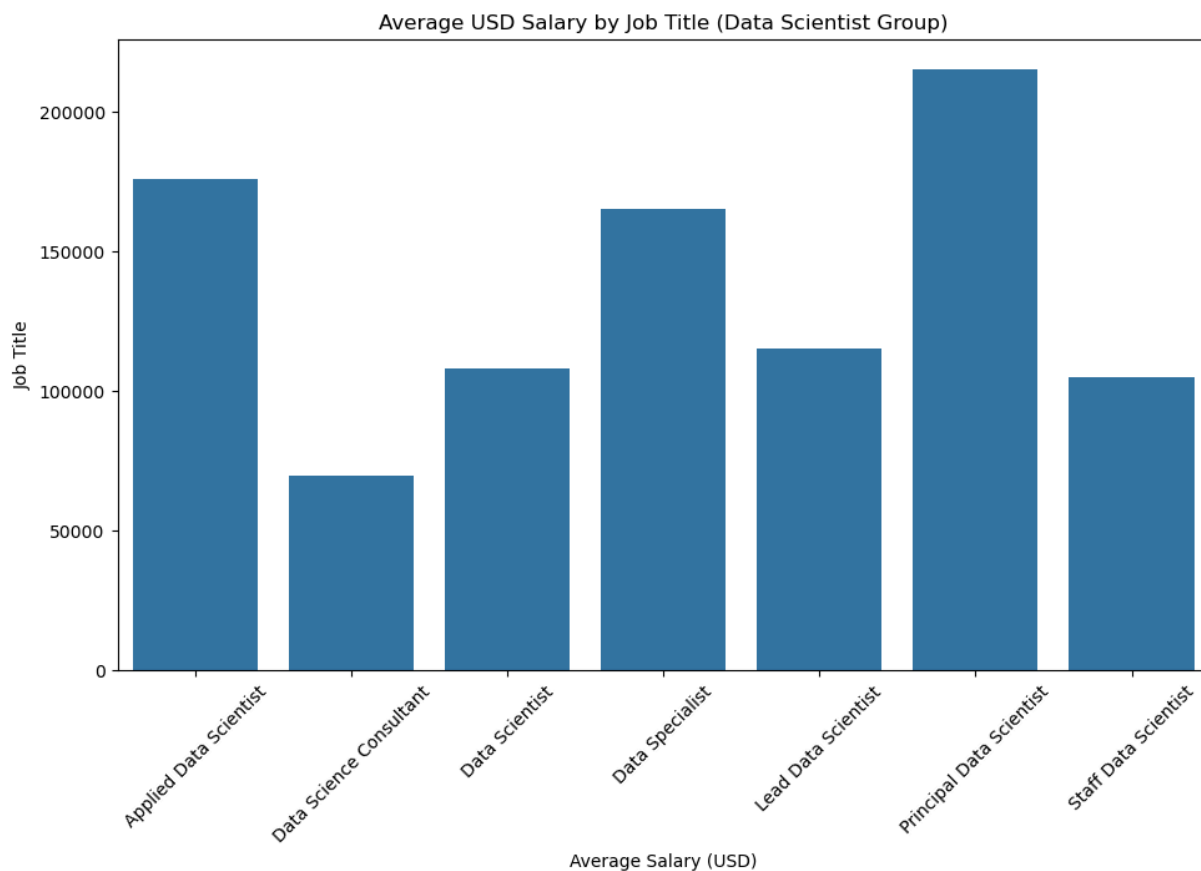
# Creating new column for job groups
title_grouped_comp_data['job_group'] = np.select(conditions, choices, default="Other")
title_grouped_comp_data[['job_title', 'job_group']].head(10)
```

Out[28]:

	job_title	job_group
0	Data Scientist	Data Scientist
1	Machine Learning Scientist	Machine Learning / AI Engineer
2	Big Data Engineer	Data Engineer / Architect
3	Product Data Analyst	Data Analyst
4	Machine Learning Engineer	Machine Learning / AI Engineer
5	Data Analyst	Data Analyst
6	Lead Data Scientist	Data Scientist
7	Data Scientist	Data Scientist
8	Business Data Analyst	Data Analyst
9	Lead Data Engineer	Data Engineer / Architect

```
In [26]: #New Graph for Job Title, filtering to job group data scientist
ds_df = title_grouped_comp_data[title_grouped_comp_data['job_group'] == "Data Scientist"]
if pd.api.types.is_categorical_dtype(ds_df['job_title']):
    ds_df['job_title'] = ds_df['job_title'].cat.remove_unused_categories() #drops empty categories
salary_means = ds_df.groupby('job_title', observed=True)['salary_in_usd'].mean().sort_values()
plt.figure(figsize=(10,6))
sns.barplot(
    data=salary_means, #using calc mean from above
    y='salary_in_usd',
    x='job_title',
)
plt.title('Average USD Salary by Job Title (Data Scientist Group)')
plt.xlabel('Average Salary (USD)')
plt.ylabel('Job Title')
plt.tight_layout()
plt.xticks(rotation=45)
plt.show()
#Much easier to read, it makes sense that positions starting with words like "Principal"
#make more money, they are positions we would expect someone to have more experience
```

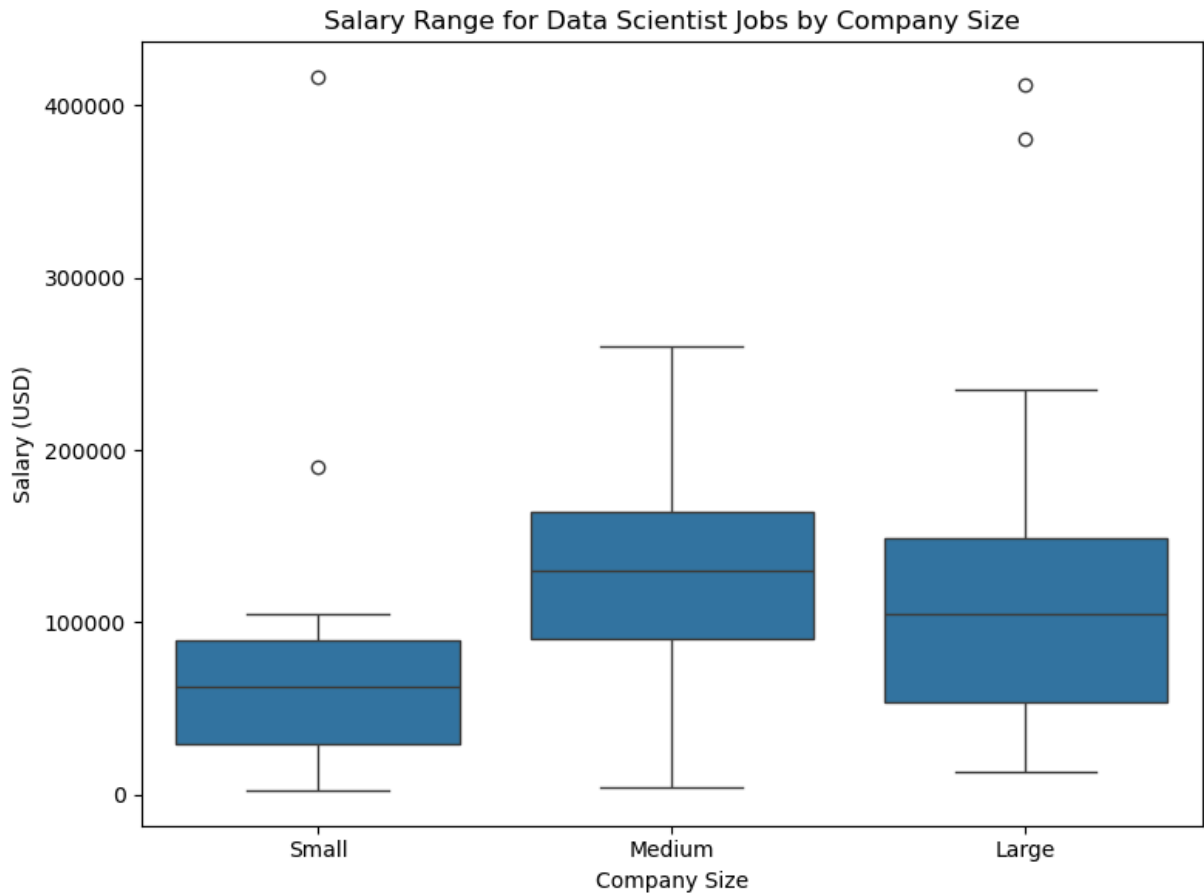
C:\Users\Lex\AppData\Local\Temp\ipykernel_5444\4257362927.py:3: DeprecationWarning: is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, pd.CategoricalDtype) instead



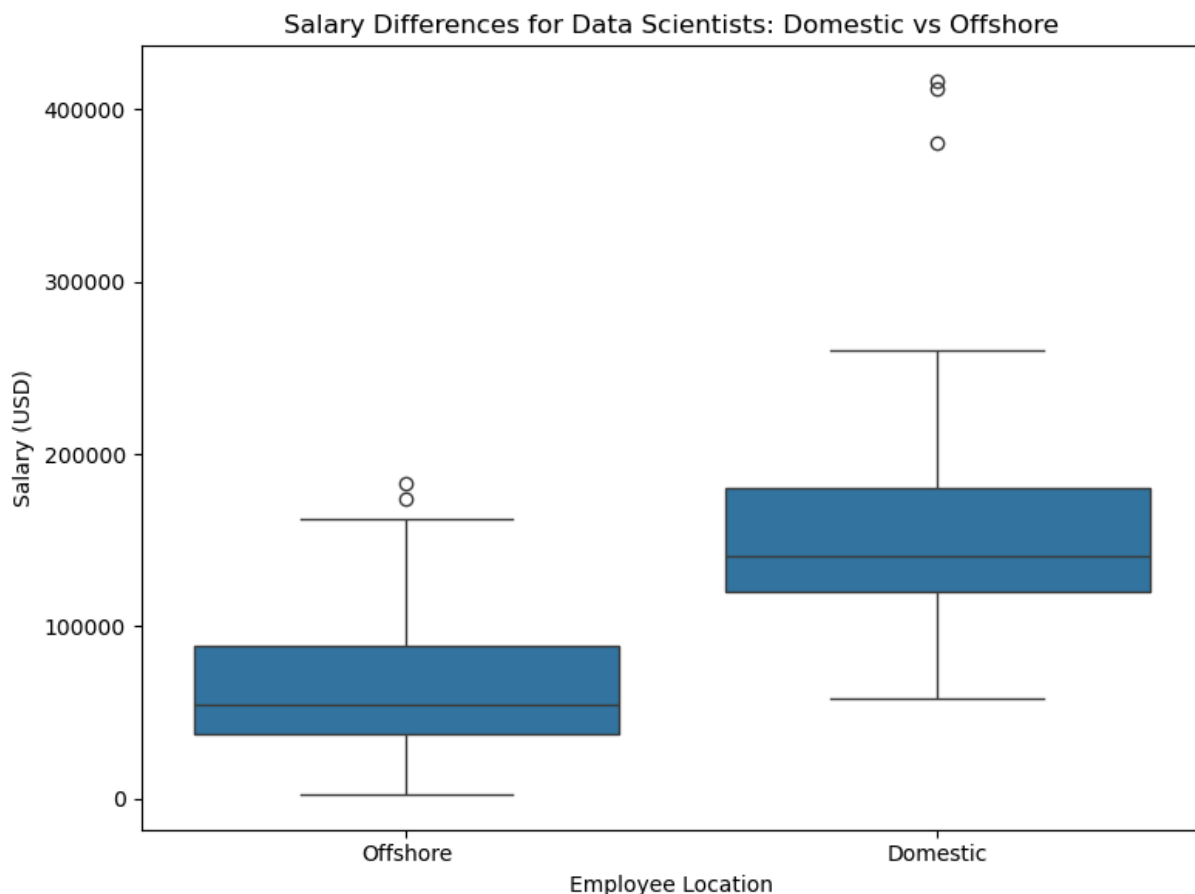
```
In [27]: # Filtering to Data Scientist group, full-time employees, and employees without a $
ds_df = title_grouped_comp_data[
    (title_grouped_comp_data['job_group'] == "Data Scientist") &
    (~title_grouped_comp_data['experience_level'].isna()) &
    (title_grouped_comp_data['salary_in_usd'] > 0)
].copy()
plt.figure(figsize=(8,6))
sns.boxplot(
    data=ds_df,
    x='experience_level',
    y='salary_in_usd',
)
plt.title('Salary Range for Data Scientist Jobs by Experience Level')
plt.xlabel('Experience Level')
plt.ylabel('Salary (USD)')
plt.tight_layout()
plt.show()
#Seeing significantly less outliers on this graph than I've seen with the graphs p
#We probably don't need an executive employee for ths position but grpah shows the
#Based on CEO's prompt we would want someone mid level to senior level for this pos
```



```
In [28]: #Graph for average data scientist salary based on company size
# Filtering to Data Scientist group, full-time employees, and employees without a $
ds_df = title_grouped_comp_data[
    (title_grouped_comp_data['job_group'] == "Data Scientist") &
    (~title_grouped_comp_data['company_size'].isna()) &
    (title_grouped_comp_data['salary_in_usd'] > 0)
].copy()
plt.figure(figsize=(8,6))
sns.boxplot(
    data=ds_df,
    x='company_size',
    y='salary_in_usd',
)
plt.title('Salary Range for Data Scientist Jobs by Company Size')
plt.xlabel('Company Size')
plt.ylabel('Salary (USD)')
plt.tight_layout()
plt.show()
#Still not a consistent difference between s, m, l like we would expect. Medium com
#This should not be a major consideration in our recommendation. The range for a sm
#There's a few outliers here, this along with the pattern shown here suggests there
```



```
In [29]: # Graph for domestic versus offshore employee salaries for data scientist grouping
ds_df = title_grouped_comp_data[
    (title_grouped_comp_data['job_group'] == "Data Scientist") &
    (~title_grouped_comp_data['salary_in_usd'].isna()) &
    (title_grouped_comp_data['salary_in_usd'] > 0)
].copy()
# Create domestic vs offshore variable
ds_df['location_group'] = ds_df['employee_residence'].apply(lambda x: 'Domestic' if
#Loops through each salary_currency listing, adds either US/offshore to new col cur
plt.figure(figsize=(8,6))
sns.boxplot(
    data=ds_df,
    x='location_group',
    y='salary_in_usd',
)
plt.title('Salary Differences for Data Scientists: Domestic vs Offshore')
plt.xlabel('Employee Location')
plt.ylabel('Salary (USD)')
plt.tight_layout()
plt.show()
#Domestic Employees cost more than offshore employees. The domestic Lower end whisk
#It would be more cost efficient for our CEO to hire an offshore employee.
#Should recommend they hire an offshore employee with a higher experience level if
#Still a couple of outliers
```



```
In [30]: #Exploring remaining outliers
ds_df = title_grouped_comp_data[
    (title_grouped_comp_data['job_group'] == "Data Scientist") &
    (title_grouped_comp_data['salary_in_usd'] > 0)
].copy()
# Create Domestic vs Offshore group variable
ds_df['location_group'] = ds_df['employee_residence'].apply(lambda x: 'Domestic' if
#Loops through each salary_currency listing, adds either US/offshore to new col cur
# Function to find outliers in a group, uses IQR formula
def find_outliers(group, column='salary_in_usd'):
    Q1 = group[column].quantile(0.25)
    Q3 = group[column].quantile(0.75)
    IQR = Q3 - Q1
    outliers = group[
        (group[column] < (Q1 - 1.5 * IQR)) |
        (group[column] > (Q3 + 1.5 * IQR))
    ]
    return outliers
# Applying to each location group
outliers_by_location = ds_df.groupby('location_group', group_keys=False).apply(find
print(outliers_by_location)
#captured all the outliers in this search and they don't look like typos. Most are
```

	work_year	experience_level	employment_type	job_title	\
63	2020	Senior Level	FT	Data Scientist	
225	2021	Executive Level	CT	Principal Data Scientist	
519	2022	Senior Level	FT	Applied Data Scientist	
257	2021	Senior Level	FT	Principal Data Scientist	
474	2022	Mid Level	FT	Data Scientist	

	salary	salary_currency	salary_in_usd	employee_residence	remote_ratio	\
63	412000	USD	412000	US	100	
225	416000	USD	416000	US	100	
519	380000	USD	380000	US	100	
257	147000	EUR	173762	DE	100	
474	140000	GBP	183228	GB	0	

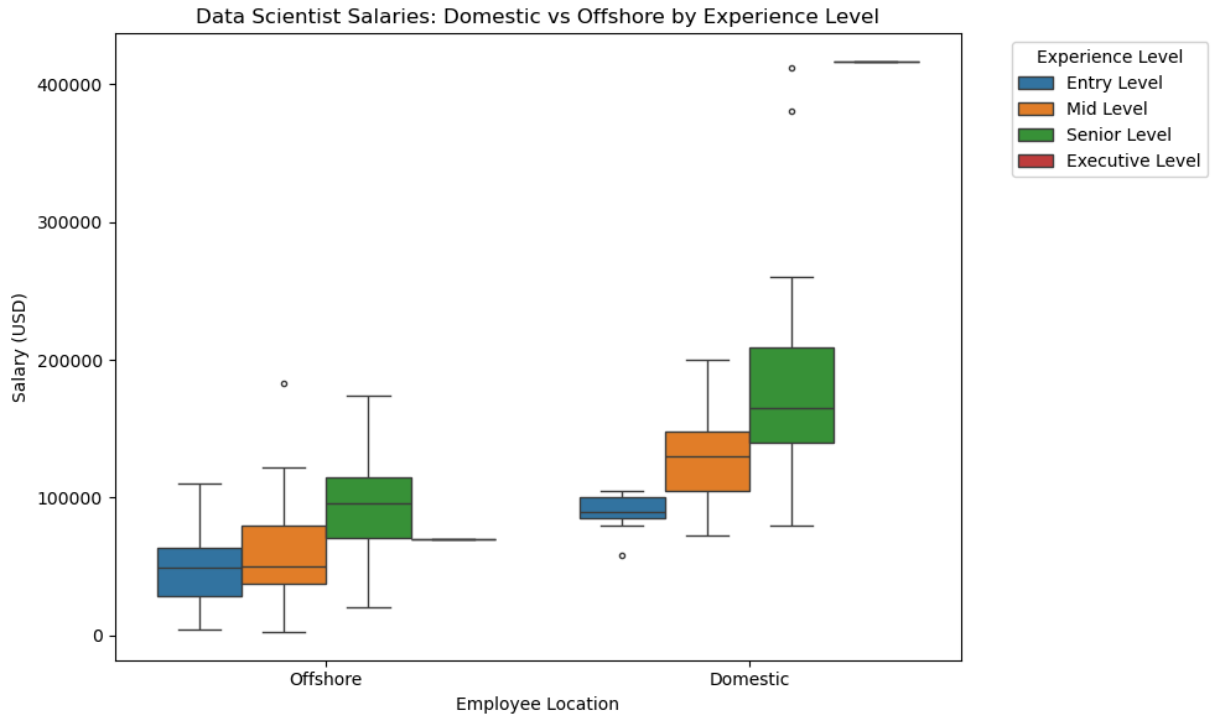
	company_location	company_size	location_group	currency_group	\
63	US	Large	Domestic	US	
225	US	Small	Domestic	US	
519	US	Large	Domestic	US	
257	DE	Medium	Offshore	Offshore	
474	GB	Medium	Offshore	Offshore	

	job_group
63	Data Scientist
225	Data Scientist
519	Data Scientist
257	Data Scientist
474	Data Scientist

C:\Users\Lex\AppData\Local\Temp\ipykernel_5444\1682948400.py:19: FutureWarning: Data FrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include\_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.

```
In [31]: #More refined table to add experience levels to better compare salary ranges
ds_df = title_grouped_comp_data[
    (title_grouped_comp_data['job_group'] == "Data Scientist") &
    (title_grouped_comp_data['salary_in_usd'] > 0)
].copy()
# Create Domestic vs Offshore group variable
ds_df['location_group'] = ds_df['employee_residence'].apply(lambda x: 'Domestic' if
#Loops through each salary_currency listing, adds either US/offshore to new col cur
plt.figure(figsize=(10,6))
sns.boxplot(
    data=ds_df,
    x='location_group',
    y='salary_in_usd',
    hue='experience_level', #color by experience levels
    fliersize=3,          # Outlier dot size
    linewidth=1
)
plt.title('Data Scientist Salaries: Domestic vs Offshore by Experience Level')
plt.xlabel('Employee Location')
plt.ylabel('Salary (USD)')
plt.legend(title='Experience Level', bbox_to_anchor=(1.05, 1), loc='upper left') #
plt.tight_layout()
plt.show()
```

#Both domestic and foreign executive levels are squished up; this needs more invest
#Other three boxplots for each grouping look as expected

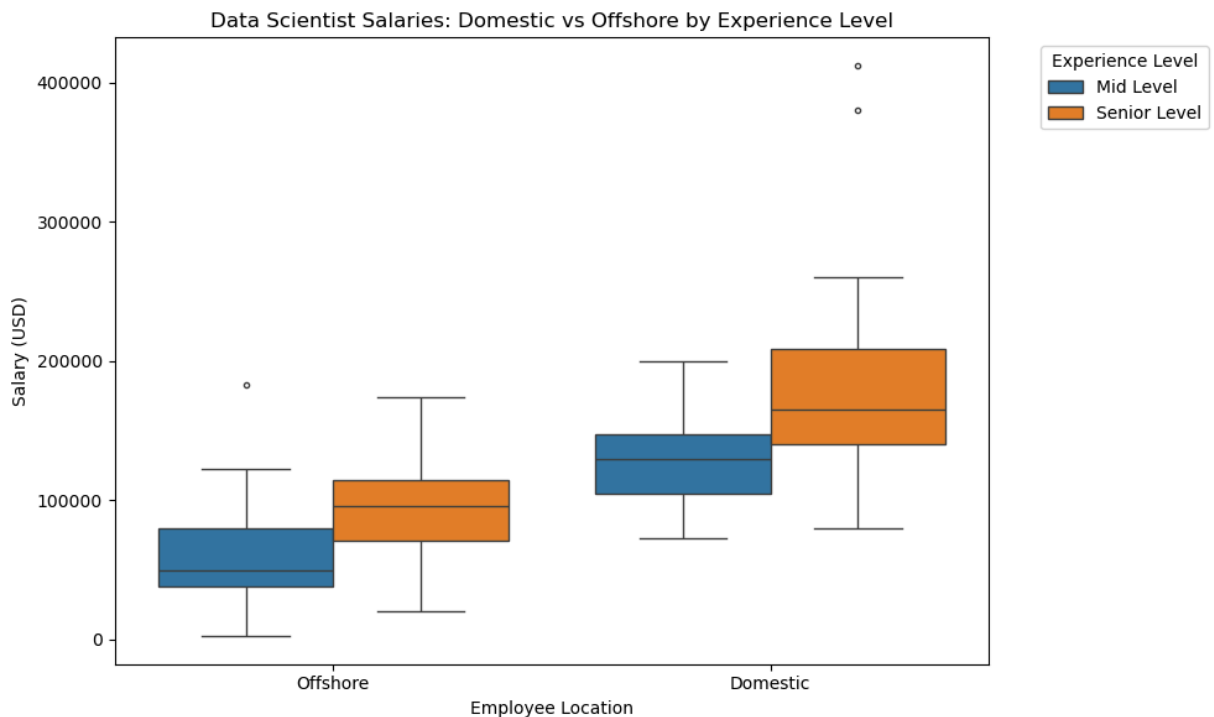


```
In [32]: #confirming why the executive boxplots are showing as just a line
ds_df = title_grouped_comp_data[title_grouped_comp_data['job_group'] == 'Data Scientist']
ds_df['location_group'] = ds_df['employee_residence'].apply(lambda x: 'Domestic' if
#Loops through each salary_currency listing, adds either US/offshore to new col cur
# Count employees by experience level within Domestic and Offshore
count_table = ds_df.groupby(['location_group', 'experience_level'], observed=True).
print(count_table)
#There's only one entry for each which is why it's showing as a line for both. Exec
```

	location_group	experience_level	count
0	Domestic	Entry Level	7
1	Domestic	Mid Level	24
2	Domestic	Senior Level	58
3	Domestic	Executive Level	1
4	Offshore	Entry Level	21
5	Offshore	Mid Level	41
6	Offshore	Senior Level	14
7	Offshore	Executive Level	1

```
In [33]: #Finalized table to add experience levels to better compare salary ranges, excluding
ds_df = title_grouped_comp_data[
    (title_grouped_comp_data['job_group'] == "Data Scientist") &
    (~title_grouped_comp_data['experience_level'].isin(["Executive Level", "Entry Level"])
    (title_grouped_comp_data['salary_in_usd'] > 0)
].copy()
# Create Domestic vs Offshore group variable
ds_df['location_group'] = ds_df['employee_residence'].apply(lambda x: 'Domestic' if
#Loops through each salary_currency listing, adds either US/offshore to new col cur
# Remove unused categories so it doesn't show on legend
ds_df['experience_level'] = ds_df['experience_level'].cat.remove_unused_categories(
plt.figure(figsize=(10,6))
```

```
sns.boxplot(
    data=ds_df,
    x='location_group',
    y='salary_in_usd',
    hue='experience_level', #color by experience levels
    fliersize=3, # Outlier dot size
    linewidth=1
)
plt.title('Data Scientist Salaries: Domestic vs Offshore by Experience Level')
plt.xlabel('Employee Location')
plt.ylabel('Salary (USD)')
plt.legend(title='Experience Level', bbox_to_anchor=(1.05, 1), loc='upper left') #
plt.tight_layout()
plt.show()
#This Looks much better
```



```
In [40]: from IPython.display import display, Markdown
# Display table title
display(Markdown("### Hiring ranges for employees based on location and experience"))
title_grouped_comp_data['employment_type'] = title_grouped_comp_data['employment_type']
ds_df = title_grouped_comp_data[
    (title_grouped_comp_data['job_group'] == "Data Scientist") &
    (title_grouped_comp_data['employment_type'] == "FT") &
    (title_grouped_comp_data['salary_in_usd'] > 0)
].copy()
ds_df = ds_df[ds_df['salary_in_usd'] >= 10000]
ds_df['currency_group'] = ds_df['salary_currency'].apply(lambda x: 'US' if x == 'US' else 'Other')
if hasattr(ds_df['experience_level'], 'cat'):
    ds_df['experience_level'] = ds_df['experience_level'].cat.remove_unused_categories()
salary_summary = (
    ds_df.groupby(['currency_group', 'experience_level'], observed=True)
    .agg(
        count=('salary_in_usd', 'size'),
        min_salary=('salary_in_usd', 'min'),
```

```

        max_salary=('salary_in_usd', 'max'),
        mean_salary=('salary_in_usd', 'mean'),
        median_salary=('salary_in_usd', 'median'),
        sd_salary=('salary_in_usd', 'std')
    )
    .reset_index()
)

# Format numbers as dollars
salary_summary[['min_salary', 'max_salary', 'mean_salary', 'median_salary', 'sd_sal
salary_summary[['min_salary', 'max_salary', 'mean_salary', 'median_salary', 'sd_sal

display(salary_summary)

```

Hiring ranges for employees based on location and experience level

	currency_group	experience_level	count	min_salary	max_salary	mean_salary	median_sal
0	Offshore	Entry Level	15	\$18,442	\$110,037	\$55,516	\$51,
1	Offshore	Mid Level	33	\$12,901	\$183,228	\$63,654	\$56,
2	Offshore	Senior Level	13	\$20,171	\$173,762	\$96,146	\$91,
3	Offshore	Executive Level	1	\$69,741	\$69,741	\$69,741	\$69,
4	US	Entry Level	9	\$13,400	\$105,000	\$75,156	\$90,
5	US	Mid Level	28	\$45,760	\$200,000	\$117,286	\$119,
6	US	Senior Level	58	\$80,000	\$412,000	\$171,959	\$165,

```

In [41]: # Display table title
display(Markdown("### Hiring ranges for employees based on location and experience
title_grouped_comp_data['employment_type'] = title_grouped_comp_data['employment_ty
ds_df = title_grouped_comp_data[
    (title_grouped_comp_data['job_group'] == "Data Scientist") &
    (title_grouped_comp_data['employment_type'] == "FT") &
    (title_grouped_comp_data['salary_in_usd'] > 0)
].copy()
ds_df['currency_group'] = ds_df['salary_currency'].apply(lambda x: 'US' if x == 'US
if hasattr(ds_df['experience_level'], 'cat'):
    ds_df['experience_level'] = ds_df['experience_level'].cat.remove_unused_categor

# Group by currency experience level
salary_summary = (
    ds_df.groupby(['currency_group', 'experience_level'], observed=True)
    .agg(
        count=('salary_in_usd', 'size'),
        min_salary=('salary_in_usd', 'min'),
        max_salary=('salary_in_usd', 'max'),
        mean_salary=('salary_in_usd', 'mean'),
        median_salary=('salary_in_usd', 'median'),
        sd_salary=('salary_in_usd', 'std')
    )
)

```

```

    )
    .reset_index()
)

# Format numbers as dollars
salary_summary[['min_salary', 'max_salary', 'mean_salary', 'median_salary', 'sd_sal
salary_summary[['min_salary', 'max_salary', 'mean_salary', 'median_salary', 'sd_sal

display(salary_summary)

```

Hiring ranges for employees based on location and experience level

	currency_group	experience_level	count	min_salary	max_salary	mean_salary	median_sal
0	Offshore	Entry Level	16	\$5,707	\$110,037	\$52,403	\$50,
1	Offshore	Mid Level	36	\$2,859	\$183,228	\$58,849	\$50,
2	Offshore	Senior Level	13	\$20,171	\$173,762	\$96,146	\$91,
3	Offshore	Executive Level	1	\$69,741	\$69,741	\$69,741	\$69,
4	US	Entry Level	10	\$4,000	\$105,000	\$68,040	\$85,
5	US	Mid Level	28	\$45,760	\$200,000	\$117,286	\$119,
6	US	Senior Level	58	\$80,000	\$412,000	\$171,959	\$165,

```

In [33]: #Table 1- Domestic versus offshore salaries
# Filter for Full-Time Data Scientists with salary and experience Level
ds_df = title_grouped_comp_data[
    (title_grouped_comp_data['job_group'] == "Data Scientist") &
    (title_grouped_comp_data['employment_type'] == "FT") &
    (~title_grouped_comp_data['experience_level'].isin(["Executive Level", "Entry
    (title_grouped_comp_data['salary_in_usd'] > 0)
].copy()
ds_df['currency_group'] = ds_df['salary_currency'].apply(lambda x: 'US' if x == 'US
#Loops through each salary_currency listing, adds either US/offshore to new col cur
# Remove unused categories so it doesn't show on Legend
ds_df['experience_level'] = ds_df['experience_level'].cat.remove_unused_categories(
# Group by currency group and experience Level
salary_summary = (
    ds_df.groupby(['currency_group', 'experience_level'], observed=True)
    .agg(
        count=('salary_in_usd', 'size'),
        min_salary=('salary_in_usd', 'min'),
        max_salary=('salary_in_usd', 'max'),
        mean_salary=('salary_in_usd', 'mean'),
        median_salary=('salary_in_usd', 'median'),
        sd_salary=('salary_in_usd', 'std')
    )
    .reset_index()
)
# Format numbers for dollar

```

```
salary_summary[['min_salary', 'max_salary', 'mean_salary', 'median_salary', 'sd_sal
salary_summary[['min_salary', 'max_salary', 'mean_salary', 'median_salary', 'sd_sal
#adds $, , seperator w/ 0 decimal places for each lambda loop
salary_summary
```

Out[33]:

	currency_group	experience_level	count	min_salary	max_salary	mean_salary	median_s
0	Offshore	Entry Level	16	\$5,707	\$110,037	\$52,403	\$5
1	Offshore	Mid Level	36	\$2,859	\$183,228	\$58,849	\$5
2	Offshore	Senior Level	13	\$20,171	\$173,762	\$96,146	\$9
3	Offshore	Executive Level	1	\$69,741	\$69,741	\$69,741	\$6
4	US	Entry Level	10	\$4,000	\$105,000	\$68,040	\$8
5	US	Mid Level	28	\$45,760	\$200,000	\$117,286	\$11
6	US	Senior Level	58	\$80,000	\$412,000	\$171,959	\$16

In [36]:

```
#Table 2- Salary rates for different company sizes
# Filter for Full-Time Data Scientists with salary and experience level
ds_df = title_grouped_comp_data[
    (title_grouped_comp_data['job_group'] == "Data Scientist") &
    (title_grouped_comp_data['employment_type'] == "FT") &
    (~title_grouped_comp_data['experience_level'].isin(["Executive Level", "Entry L
    (title_grouped_comp_data['salary_in_usd'] > 0)
].copy()
ds_df['currency_group'] = ds_df['salary_currency'].apply(lambda x: 'US' if x == 'US
# Remove unused categories so it doesn't show on Legend
ds_df['experience_level'] = ds_df['experience_level'].cat.remove_unused_categories(
# Group by currency group and experience level
salary_summary = (
    ds_df.groupby(['company_size', 'experience_level'], observed=True)
    .agg(
        count=('salary_in_usd', 'size'),
        min_salary=('salary_in_usd', 'min'),
        max_salary=('salary_in_usd', 'max'),
        mean_salary=('salary_in_usd', 'mean'),
        median_salary=('salary_in_usd', 'median'),
        sd_salary=('salary_in_usd', 'std')
    )
    .reset_index()
)
# Format numbers for dollar
salary_summary[['min_salary', 'max_salary', 'mean_salary', 'median_salary', 'sd_sal
salary_summary[['min_salary', 'max_salary', 'mean_salary', 'median_salary', 'sd_sal
#adds $, , seperator w/ 0 decimal places for each lambda loop
salary_summary
```

Out[36]:

	company_size	experience_level	count	min_salary	max_salary	mean_salary	median_sal
0	Small	Mid Level	12	\$2,859	\$82,500	\$35,957	\$33,
1	Small	Senior Level	3	\$87,738	\$190,000	\$122,992	\$91,
2	Medium	Mid Level	24	\$33,808	\$200,000	\$101,014	\$96,
3	Medium	Senior Level	46	\$77,684	\$260,000	\$156,186	\$147,
4	Large	Mid Level	28	\$31,615	\$160,000	\$90,956	\$90,
5	Large	Senior Level	22	\$20,171	\$412,000	\$166,816	\$161,

In [37]:

```
#T-Test for salaries in USD for domestic versus offshore groups
# Filter for Data Scientist, full-time, salary > 0, exclude Executive and entry Lev
ds_df = title_grouped_comp_data[
    (title_grouped_comp_data['job_group'] == "Data Scientist") &
    (title_grouped_comp_data['employment_type'] == "FT") &
    (~title_grouped_comp_data['experience_level'].isin(["Executive Level", "Entry L
    (title_grouped_comp_data['salary_in_usd'] > 0)
].copy()
ds_df['location_group'] = ds_df['employee_residence'].apply(lambda x: 'Domestic' if
# Separate the two groups domestic & offshore
domestic_salaries = ds_df.loc[ds_df['location_group'] == 'Domestic', 'salary_in_usd']
offshore_salaries = ds_df.loc[ds_df['location_group'] == 'Offshore', 'salary_in_usd']
# Perform Welch's t-test- not assuming equal variance
t_stat, p_value = ttest_ind(domestic_salaries, offshore_salaries, equal_var=False)
print("Welch Two Sample t-test Results")
print(f"t-statistic: {t_stat:.3f}") #f-string; ":"= command, for fixed decimal form
print(f"p-value: {p_value:.3e}") #f-string; .3e is command for scientific notation
print(f"Mean Domestic Salary: ${domestic_salaries.mean():.2f}")
#f-string; ":" starts command to add , sperator for num, ".2f" = fixed decimal form
print(f"Mean Offshore Salary: ${offshore_salaries.mean():.2f}")
```

Welch Two Sample t-test Results
t-statistic: 10.664
p-value: 1.675e-19
Mean Domestic Salary: \$159,245.06
Mean Offshore Salary: \$69,022.87

In [45]:

```
#T test mid Level
ds_df = title_grouped_comp_data[
    (title_grouped_comp_data['job_group'] == "Data Scientist") &
    (title_grouped_comp_data['employment_type'] == "FT") &
    (title_grouped_comp_data['experience_level'].isin(["Mid Level"])) &
    (title_grouped_comp_data['salary_in_usd'] > 0)
].copy()
ds_df['location_group'] = ds_df['employee_residence'].apply(lambda x: 'Domestic' if
# Separate the two groups domestic & offshore
domestic_salaries = ds_df.loc[ds_df['location_group'] == 'Domestic', 'salary_in_usd']
offshore_salaries = ds_df.loc[ds_df['location_group'] == 'Offshore', 'salary_in_usd']
# Perform Welch's t-test- not assuming equal variance
t_stat, p_value = ttest_ind(domestic_salaries, offshore_salaries, equal_var=False)
print("Welch Two Sample t-test Results for Mid Level Employees")
```

```
print(f"t-statistic Mid Level: {t_stat:.3f}") #f-string; ":"= command, for fixed de
print(f"p-value Mid Level: {p_value:.3e}") #f-string; .3e is command for scientific
print(f"Mean Domestic Mid Level Salary: ${domestic_salaries.mean():.2f}")
#f-string; ":" starts command to add , sperator for num, ".2f" = fixed decimal form
print(f"Mean Offshore Mid Level Salary: ${offshore_salaries.mean():.2f}")
```

Welch Two Sample t-test Results for Mid Level Employees

t-statistic Mid Level: 7.798

p-value Mid Level: 1.645e-10

Mean Domestic Mid Level Salary: \$126,052.08

Mean Offshore Mid Level Salary: \$59,433.40

```
In [46]: #T test Senior Level
ds_df = title_grouped_comp_data[
    (title_grouped_comp_data['job_group'] == "Data Scientist") &
    (title_grouped_comp_data['employment_type'] == "FT") &
    (title_grouped_comp_data['experience_level'].isin(["Senior Level"])) &
    (title_grouped_comp_data['salary_in_usd'] > 0)
].copy()
ds_df['location_group'] = ds_df['employee_residence'].apply(lambda x: 'Domestic' if
# Separate the two groups domestic & offshore
domestic_salaries = ds_df.loc[ds_df['location_group'] == 'Domestic', 'salary_in_usd']
offshore_salaries = ds_df.loc[ds_df['location_group'] == 'Offshore', 'salary_in_usd']
# Perform Welch's t-test- not assuming equal variance
t_stat, p_value = ttest_ind(domestic_salaries, offshore_salaries, equal_var=False)
print("Welch Two Sample t-test Results for Senior Level Employees")
print(f"t-statistic Senior Level: {t_stat:.3f}") #f-string; ":"= command, for fixed
print(f"p-value Senior Level: {p_value:.3e}") #f-string; .3e is command for scienti
print(f"Mean Domestic Senior Level Salary: ${domestic_salaries.mean():.2f}")
#f-string; ":" starts command to add , sperator for num, ".2f" = fixed decimal form
print(f"Mean Offshore Senior Level Salary: ${offshore_salaries.mean():.2f}")
```

Welch Two Sample t-test Results for Senior Level Employees

t-statistic Senior Level: 5.409

p-value Senior Level: 1.228e-05

Mean Domestic Senior Level Salary: \$173,221.05

Mean Offshore Senior Level Salary: \$96,421.36

```
In [ ]: #T is large suggesting that there is a large difference between the two groups. P-v
#so that means that there is a very small chance that there is not a difference bet
#The difference in the means of the two groups is statistically significant. I can
#CEO that it would be cheaper to higher an offshore employee; based on these t-test
#reason to believe that the cost would be much cheaper. Cost of the employee could
#reduced by half.
```

```
In [44]: #Salary Ranges
ds_df = title_grouped_comp_data[
    (title_grouped_comp_data['job_group'] == "Data Scientist") &
    (title_grouped_comp_data['employment_type'] == "FT") &
    (title_grouped_comp_data['experience_level'].isin(["Mid Level", "Senior Level"])
    (title_grouped_comp_data['salary_in_usd'] > 0)
].copy()
# Create Domestic vs Offshore grouping
ds_df['currency_group'] = ds_df['salary_currency'].apply(
    lambda x: 'US' if x == 'USD' else 'Offshore'
)
ds_df['experience_level'] = ds_df['experience_level'].cat.remove_unused_categories(
```

```

# Group and calculate percentiles
salary_summary = (
    ds_df.groupby(['currency_group', 'experience_level'], observed=True)
    .agg(
        count=('salary_in_usd', 'size'),
        Quartile_One_salary=('salary_in_usd', lambda x: x.quantile(0.25)),
        median_salary=('salary_in_usd', 'median'),
        Quartile_Three_salary=('salary_in_usd', lambda x: x.quantile(0.75))
    )
    .reset_index()
)
# Format as dollars
salary_summary[['Quartile_One_salary', 'median_salary', 'Quartile_Three_salary']] =
    salary_summary[['Quartile_One_salary', 'median_salary', 'Quartile_Three_salary']]
    .map(lambda x: f"${x:,.0f}") #adds $, , seperator w/ 0 decimal places for each
salary_summary

```

Out[44]:

	currency_group	experience_level	count	Quartile_One_salary	median_salary	Quartile_Th
--	----------------	------------------	-------	---------------------	---------------	-------------

0	Offshore	Mid Level	36	\$35,699	\$50,524	
1	Offshore	Senior Level	13	\$68,428	\$91,237	
2	US	Mid Level	28	\$99,075	\$119,000	
3	US	Senior Level	58	\$140,000	\$165,000	

